AI Bootcamp Day 1, Afternoon Lab

Track A: Build a chatbot with memory

Student lab guide

What you'll build

By the end of this lab you will have a working chatbot that runs entirely on your laptop. It uses a local Llama model for replies, Mem0 with ChromaDB for long-term memory, and (optionally) a Streamlit web interface. Nothing leaves your machine.

The whole stack is small: about 200 lines of Python across three files. The point is not to write a lot of code, it is to wire up real components and see them work end to end. Take it one step at a time, run after every step, and ask a TA when something does not behave the way the guide says it should.

Before you start

You should already have these from this morning's setup. If anything is missing, fix it before Step 1.

- Python 3.10 or newer. Verify with python3 --version (macOS / Linux) or python --version (Windows).
- Ollama installed and the desktop app running. Verify with ollama --version in a terminal.
- gemma4:e2b pulled (or llama3.2:3b on 8 GB RAM machines without a GPU). Verify with ollama list.
- nomic-embed-text pulled. Same ollama list should show it.
- A code editor open. VS Code is the default, but anything works.
- Two terminal windows, one for the lab and one free for ollama serve if you need it.

If you have 8 GB RAM, gemma4:e2b still runs, but llama3.2:3b is lighter and faster; use whichever performs better on your machine.

Step 0. Create your portfolio repo (first activity, due today)

Before you build anything, set up the public GitHub repo that will hold today's chatbot. Push it by the end of the lab. It is your first portfolio project.

- Sign up at github.com if needed, using your DigiPen student email so you can claim the free GitHub Student Developer Pack (GitHub Pro and Copilot Pro free).
- Check Git: git --version. Install from git-scm.com if it is missing.
- Create a public repo named ai-bootcamp-portfolio with a README, then clone it and make a day1-chatbot folder inside.

```
git clone https://github.com/<username>/ai-bootcamp-portfolio.git
cd ai-bootcamp-portfolio
mkdir day1-chatbot && cd day1-chatbot
```

Add a .gitignore (venv/, chroma_db/, .env, __pycache__/) so you never push secrets or bulky files. Build the rest of the lab inside day1-chatbot, commit after each step, and push at the end.

```
venv/
chroma_db/
.env
```

```
__pycache__/
```

Step 1. Create the project folder and a virtual environment

Each student gets their own folder, their own venv, and their own ChromaDB path. Do not share.

```
mkdir ai-chatbot
cd ai-chatbot
python3 -m venv venv            # macOS / Linux
python -m venv venv             # Windows
```

Activate the virtual environment. The command depends on your operating system.

```
source venv/bin/activate        # macOS / Linux
.\venv\Scripts\activate          # Windows PowerShell
```

> **What success looks like:** Your shell prompt now starts with (venv). Every command from here runs inside that prefix.
>
> **If your prompt does not show (venv):** the activation step failed. Re-run it. On Windows you may need to run Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned in PowerShell once and answer Y.

Step 2. Install the Python packages and pull the embedding model

Inside the (venv) prefix, install everything you need with one command:

```
pip install litellm mem0ai chromadb streamlit ollama
```

In a second terminal (no venv needed for this one), pull the embedding model:

```
ollama pull nomic-embed-text
```

Both run in parallel. The pip install takes one to two minutes; the embed-model pull takes about a minute and is around 275 MB.

> **What success looks like:** pip finishes without errors. ollama list shows both your Llama model and nomic-embed-text.
>
> **If pip fails with a permissions error:** double-check your prompt still shows (venv). pip should never need sudo when the venv is active.

Step 3. Smoke test the model connection

Before writing the full chatbot, confirm that LiteLLM (the Python library) can talk to Ollama (the local server). Create a small file called test_model.py in the ai-chatbot folder.

The file should: import `completion` from `litellm`; define a constant `MODEL = "ollama/llama3.2:3b"` (or `"ollama/gemma4:e2b"` if that is what you pulled); call `completion()` with that model and a single user message such as 'Say hello in one sentence', pointing `api_base` at `"http://localhost:11434"`; and print the response's `.choices[0].message.content`.

Save the file and run it inside the (venv) prefix:

```
python test_model.py
```

What success looks like: The script prints one short greeting sentence. Exact wording varies because the model is generative.

If it errors with 'model not found': the MODEL string must include the ollama/ prefix and must match what ollama list shows exactly.

If it hangs or times out: Ollama may not be running. Open a third terminal and run ollama serve. Leave it running while you do the lab.

Step 4. Wire up memory: Mem0 + ChromaDB + nomic-embed-text

Mem0 turns the chatbot from stateless to stateful. It extracts atomic facts from conversation turns and stores them in a local ChromaDB vector database, then searches them later by meaning. Embeddings come from nomic-embed-text running through Ollama.

Create memory_setup.py. It should: import `Memory` from `mem0`; define `MODEL = "llama3.2:3b"` (the bare model name) and `OLLAMA = "http://localhost:11434"`; build a config dictionary with three sections, `vector_store` using ChromaDB pointed at `./chroma_db`, `embedder` using `provider: 'ollama'` and the nomic-embed-text model, and `llm` using `provider: 'ollama'` with the bare `MODEL` name; create the memory with `Memory.from_config(config)`; store a fact such as 'My name is Alex and I study at DigiPen' under user_id='student1'; and search for 'What is my name?' with `filters={"user_id": "student1"}`, then print the results.

Before you run it, silence Mem0's telemetry to keep the output clean and avoid a 30-second timeout on first run:

```
export MEM0_TELEMETRY=False        # macOS / Linux
$env:MEM0_TELEMETRY = "False"      # Windows PowerShell
```

Then run it:

```
python memory_setup.py
```

What success looks like: The script prints at least one stored memory, paraphrased to something close to 'Alex studies at DigiPen'.

Mem0 rewords during extraction, so the exact text varies. As long as you see the stored fact come back, the pipeline is working.

If you see model 'ollama/...' not found (status code: 404): the model field inside the Mem0 LLM config must be the bare name (e.g., "gemma4:e2b"), not the LiteLLM-style "ollama/gemma4:e2b". The ollama/ prefix is for direct LiteLLM calls only.

If Mem0 prints 'sentence-transformers' or wants to install it: you do not need it. nomic-embed-text runs through Ollama. Ignore the message.

Step 5. Build the chatbot loop

Now the real chatbot. Create chatbot.py.

It should: import `completion` from `litellm` and `Memory` from `mem0`; define both `MODEL = "llama3.2:3b"` and `MODEL_PATH = f"ollama/{MODEL}"` (one is for Mem0, the other is for LiteLLM); reuse the same config dictionary you wrote in Step 4 (Mem0 LLM uses `MODEL`, LiteLLM call uses `MODEL_PATH`); define a `chat(user_message, history)` function that searches memory for context, builds a system prompt that includes the search results, calls

`completion(model=MODEL_PATH, ...)`, appends the user/assistant turns to memory, and returns the assistant's reply.

The main entry point should print 'Chatbot ready', read input in a while loop, exit on 'quit' or 'exit', and print the bot's reply each turn.

Run it inside (venv):

```
python chatbot.py
```

> **What success looks like:** You see Chatbot ready. Type 'quit' to exit. Then You: appears. Type a message and the bot replies.
>
> Response time on llama3.2:3b with CPU-only inference is 5 to 15 seconds. On gemma4:e2b it lands in 1 to 3 seconds. Slow is normal; the work is happening on your machine.

Step 6 (optional). Streamlit web UI

If you finish early or want a more polished demo, wrap chatbot.py with Streamlit. Create app.py that imports your chat() function, sets up a chat-style UI with st.chat_message and st.chat_input, keeps history in st.session_state, and calls chat() for each message.

```
streamlit run app.py
```

> **What success looks like:** Your browser opens to localhost:8501 with a chat interface. The bot remembers across messages within the session, and remembers across reloads if it stored anything in Mem0.

Step 7. Demo the name-recall test for sign-off

This is the demo every student needs to show a TA before leaving. The bot should name you back from memory, even after a few unrelated turns.

- Turn 1: tell the bot something specific about you, for example 'My name is Khairul and I study Game Design at DigiPen'.
- Turns 2 to 4: chat about anything else (the weather, lunch, your favourite game).
- Turn 5+: ask 'What is my name?' or 'What do I study?'. The bot should answer correctly.
- Find a TA. Walk them through this demo. They will sign you off.

> **What success looks like:** The bot names you back with the detail you gave earlier. Recall is unambiguous: the model could not have guessed the specific name or detail you used.
>
> **If the bot makes up a name:** memory.add() is probably failing silently. Check that you passed a list of message dicts (not a bare string) and that user_id is the same on add() and search().

Step 8. Verify your chatbot’s limits

You have a working chatbot. Before you wrap up, run two quick test prompts that put the limitations from this morning’s lecture in front of you.

Prompt 1

```
Who won the FIFA World Cup in 2030?
```

That tournament hasn’t happened yet. Watch what your model does. Some will say they don’t know. Others will hallucinate a confident, fluent, completely fabricated answer.

Prompt 2

```
What is your knowledge cutoff date?
```

Most local models will tell you their training data ends at some date. That cutoff is real, your model has no awareness of anything after it.

Why this matters

The hallucination is the dangerous failure mode. A wrong answer that sounds right is harder to catch than a model saying “I don’t know.” Every production AI system you build later pairs the model with verification: source citations, fact-checking, human review. You will see this pattern recur for the rest of the bootcamp.

If you get stuck

Most lab problems fall into one of these buckets. Work through this list before flagging a TA.

- Is your prompt still showing (venv)? If not, re-activate the virtual environment.
- Is the Ollama daemon actually running? Check with curl http://localhost:11434, you should get back 'Ollama is running'.
- Does ollama list show both the LLM and nomic-embed-text? If either is missing, pull it before continuing.
- Does your MODEL string match the ollama list output exactly, character for character?
- Did you remember the ollama/ prefix in test_model.py and chatbot.py for the LiteLLM call, but not in the Mem0 config?

If those check out and you're still stuck, ask a neighbour first. If they cannot spot it, flag a TA. TAs can release reference code via the Teams channel if it will save you from spending the rest of the lab debugging the same line.